Placement Portal Application - V2

Institutes require efficient systems to manage campus recruitment activities involving companies and students. Currently, many institutes rely on spreadsheets, emails, or manual coordination, which makes it difficult to manage company approvals, placement drives, student registrations, and application tracking.

You are required to build a **Placement Portal Application (PPA)** web application that allows **Admin (Institute), Companies, and Students** to interact with the system based on their roles.

Frameworks to be used

These are the mandatory frameworks on which the project has to be built.

- Flask for API
- VueJS for UI
- VueJS Advanced with CLI (only if required, not necessary)
- Jinja2 templates if using CDN **only for entry point** (not to be used for UI)
- Bootstrap for HTML generation and styling (no other CSS framework is allowed)
- SQLite for database (no other database is permitted)
- Redis for caching
- Redis and Celery for batch jobs

Note:
All demos should be possible on your local machine.
Any other framework/library will not be allowed apart from those given above.
The database must be created programmatically (via table creation or model code). Manual database creation (e.g., DB Browser for SQLite) is NOT allowed.

Roles & Functionalities

This platform has **three** roles:

1. **Admin (Institute Placement Cell)**
    a. Admin is the pre-existing superuser of the application.
    b. Can approve or reject company registrations.
    c. Can approve or reject placement drives created by companies.
    d. Can view and manage all students, companies, and placement drives.
    e. Can search for students or companies.
    f. Can blacklist or deactivate companies and students if required.
    g. Can view reports and placement statistics.
2. **Company**
    h. Can register their company profile on the platform.
    i. Can create placement drives only after admin approval.
    j. Can view student applications for their placement drives.
    k. Can shortlist students and update application status.
    l. Can schedule interviews and update final selection results.
3. **Student**
    m. Can register, log in, and update their profile.
    n. Can view approved placement drives.
    o. Can apply for placement exams/drives created by companies.
    p. Can view application status and placement history.

Key Terminologies

1. **Admin (Institute):** A user with the highest level of access who manages companies, students, and placement drives.
2. **Company:** An organization registered in the system that conducts placement drives and recruits students.
3. **Student:** A user who applies for placement drives and participates in recruitment processes.
4. **Placement Drive:** A recruitment event created by a company.

    Attributes:

    a. Drive ID
    b. Company ID
    c. Job Title
    d. Job Description
    e. Eligibility Criteria (branch, CGPA, year)
    f. Application Deadline
    g. Status (Pending / Approved / Closed)
    h. Extra fields etc.
5. **Application:** A record of a student applying to a placement drive.

    Attributes:

    i. Application ID
    j. Student ID
    k. Drive ID
    l. Application Date
    m. Status (Applied / Shortlisted / Selected / Rejected)
    n. Extra fields etc.

q. HR Contact
r. Website
s. Approval Status
t. Extra fields etc.

Note: The above tables and fields are not exhaustive. Students can add more tables and fields as per their requirements.

Similar Apps

https://internshala.com/
https://www.indeed.com/
https://angel.co/

Application Wireframe

Placement Portal Application

Note:
The provided wireframe is intended **only to illustrate the application's flow** and demonstrate what should appear when a user navigates between pages.

- **Replication of the exact views is NOT mandatory.**
- Students are encouraged to work on their front-end ideas and designs while maintaining the application's intended functionality and flow.

Core Features

- **Admin and User Login Functionality**
  - Login/register form for students and companies.
  - Login only for admin (no admin registration), and this application should have only one admin identified by its role.
  - Role-based access control and authentication using **Flask security (session or token)** or **JWT based Token**.
  - A unified user model to differentiate all types of user roles.

- **Admin Functionalities**
  - Admin dashboard showing:
    - Total number of students
    - Total number of companies
    - Total number of placement drives
  - Admin must pre-exist and be created programmatically after database creation. **[No admin registration allowed]**
  - Admin can approve or reject company registrations.
  - Admin can approve or reject placement drives.
  - Admin can view all student applications.
  - Admin can search companies and students.
  - Admin can blacklist or deactivate companies/students.

- **Company Functionalities**
  - Companies can register their company profile.
  - Company dashboard showing:
    - Company details
    - Created placement drives
    - Number of applicants per placement drive
  - Companies can create placement drives (only after approval from admin).
  - Companies can view and manage student applications.
  - Companies can shortlist students and update selection status.

- **Student Functionalities**
  - Students have to self-register and login.
  - Student dashboard showing:
    - All approved placement drives
    - Eligibility-based filtering or Searching
  - Students can apply for placement drives.
  - Students can view application status.
  - Students can view past placement history.
  - Students can edit their profile and upload resume.

- **Backend Jobs**

  a. Scheduled Job - Daily reminders - The application should send daily reminders to users on g-chat using Google Chat Webhooks or SMS or mail

  - Send reminders to students about upcoming application deadlines.
  - Can be sent via email, SMS, or chat webhook.
  - Runs daily at a chosen time.

  b. Scheduled Job - Monthly Activity Report - Devise a monthly report for the Admin created using HTML and sent via mail.

  - Generate monthly placement activity reports for the institute.
  - Includes:
    - Number of drives conducted
    - Number of students applied and selected
  - Report generated on the first day of every month.
  - Sent to admin via email.

  c. User Triggered Async Job – Export Applications as CSV

  - Students can export their placement application history.
  - CSV should include:
    - Student ID
    - Company Name
    - Drive Title
    - Application Status
    - Dates
  - Export is triggered from the student dashboard.
  - This should trigger a batch job, and send an alert once done.

- **Performance and Caching**

- Prevent students from applying multiple times to the same drive.
- Update application status dynamically.
- Eligibility validation before allowing applications.
- Admin and students should be able to search companies and drives.
- Maintain complete placement history for each student.

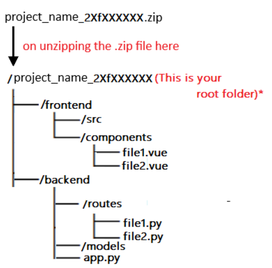
Recommended and/or Optional functionalities

1. Well-designed PDF reports for Monthly activity reports (Students can choose between HTML and PDF reports)
2. External APIs/libraries for creating charts, e.g. ChartJS
3. Single Responsive UI for both Mobile and Desktop
   a. Unified UI that works across devices
   b. Add to desktop feature
4. Implementing frontend validation on all the form fields using HTML5 form validation or JavaScript
5. Implementing backend validation within your APIs
6. Provide styling and aesthetics to your application by creating a beautiful and responsive front end using simple CSS or Bootstrap
7. Implement either a **dummy offer letter generator for the company side** or an **ATS-style resume checker for institutes, students, or companies**.
8. Any additional feature you feel is appropriate for the application.

Evaluation

1. Students have to create and submit a project report (not more than 5 pages) on the portal, along with the actual project submission
2. The report must include the following things;
   1. Student details
   2. Project details, including the question statement and how you approached the problem statement
   3. AI/LLM Declaration
   4. Frameworks and libraries used
   5. ER diagram of your database, including all the tables and their relations
   6. API resource endpoints (if any)
   7. Drive link of the presentation video
3. **If a student has used any form of AI/LLM for the project, you will need to mention the extent of use in your report, (click here for the doc)**
4. **You can find a template to create the project report (click here for project report demo)**

Possible folder structure:



5. All code is to be submitted on the portal in a single zip file (zipping instructions are given in the project document - Project Doc T12026
6. Video Presentation Guidelines (Advised):
   1. A short intro (not more than 30 sec)
   2. How did you approach the problem statement? (30 sec)
   3. Highlight key features of the application (90 sec)
   4. Any additional feature(s) implemented other than core requirements (30 sec)

   Note:
   1. The final video **must not exceed 5-10 minutes**.
   2. Keeping your video feed on during recording (like in a screencast) is optional but recommended.
7. The video must be uploaded on the student drive with **access to anyone with the link** and the link must be included in the report:
   1. This will be viewed during or before the viva, so it should be a clear explanation of your work.
8. Viva: After the video explanation, you are required to give a demo of your work, and answer any questions that the examiner asks:
   1. This includes making changes as requested and running the code for a live demo.
   2. Other questions may be unrelated to the project itself, but are relevant to the course.

Instructions

1. This is a live document and will be updated with more details (wireframe)
2. We will freeze the problem statement on or before **10/02/2026**, beyond which any modifications to the statement will be communicated via proper announcements.
3. The project has to be submitted as a single zip file.